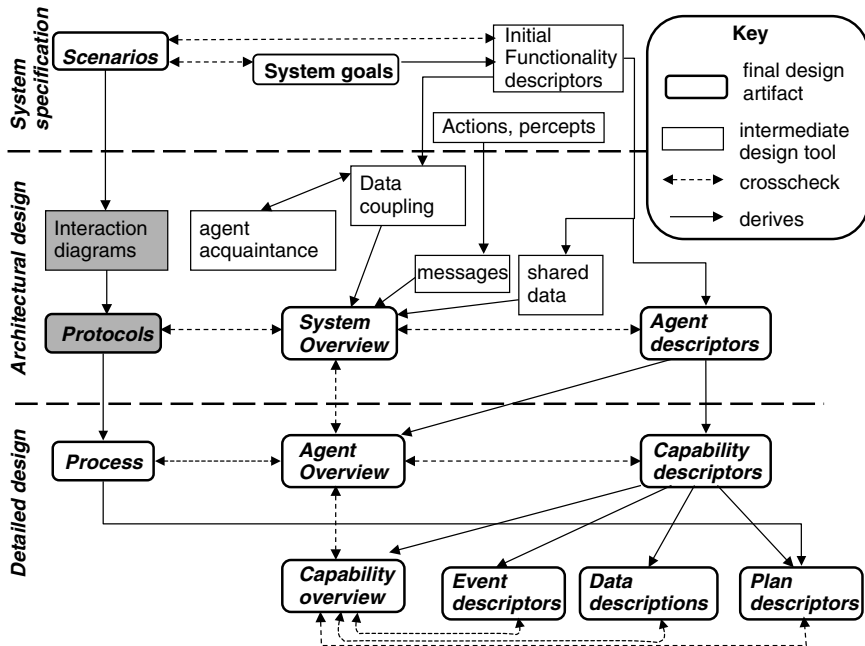


Architectural Design: Specifying the Interactions



Phases, artifacts and relationships in the design process

Once the agent types are decided, the next aspect of the architectural design is to specify the interaction between agents, capturing the *dynamic* aspects of the system. This builds on both the agent types and also on the scenario descriptors from the System Specification phase.

The development of the specification of the interaction between agents (step 2 in Figure 5.1) has the following stages:

- 1. Develop interaction diagrams from use case scenarios.
- 2. Generalize interaction diagrams to interaction protocols.
- 3. Develop protocol and message descriptors.

Like use case scenarios, interaction diagrams describe only a single example of the system’s behaviour. The fully specified *interaction protocols* are then developed from these and are the final design artifact.

6.1 INTERACTION DIAGRAMS FROM SCENARIOS

Interaction diagrams are borrowed directly from object-oriented design, but show interaction between agents rather than objects. The process for developing interaction diagrams is to take the scenarios developed in the specification phase and to build corresponding interaction diagrams.

This involves (a) replacing each functionality with the agent that includes it; (b) inserting a communication between agents where it is needed and then (c) expressing the result as an interaction diagram. This process is summarized in Figure 6.1.

For example, suppose we have a scenario that includes the following steps:

- 1. GOAL: Update user record (functionality Student Records)
- 2. GOAL: Check money owing (functionality Student Finances)
- 3. GOAL: Request graduation (functionality Graduation)

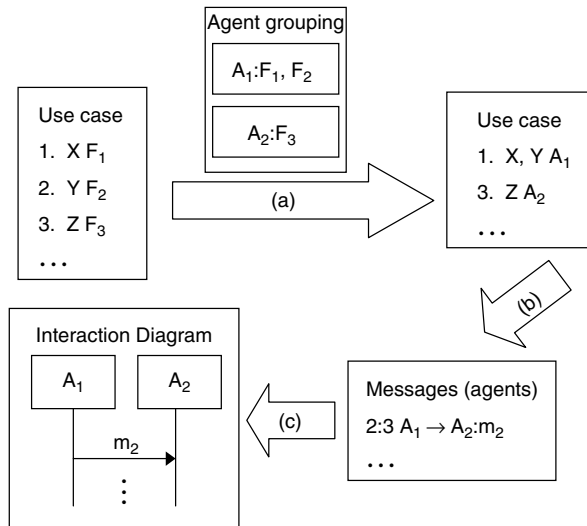


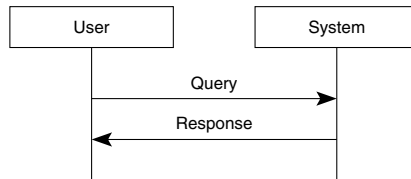
Figure 6.1 From use case scenarios to interaction diagrams

Suppose that the first two functionalities (Student Records and Student Finances) are both in an agent 'Student Manager' and that Graduation is in an agent 'Awards'. Then the first two steps are done by Student Manager and the third step by Awards. The Awards agent needs to be told (by Student Manager) when to request a graduation, thus there is a message from Student Manager to Awards after step 2 (and before step 3).

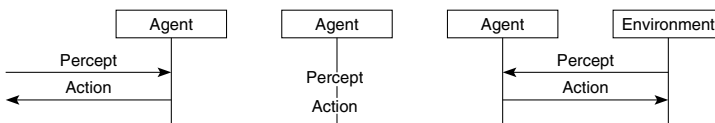
Of the three steps depicted in Figure 6.1, the first and third are quite simple. Step (a) involves replacing functionalities with agents on the basis of the grouping of functionalities into agents. If a functionality F has been grouped into agent type A , then steps in a use case scenario that are done by the functionality F are considered to be done by the agent A .

The result of step (b) is a list of messages. Step (c) involves a simple change of notation from this list of messages (with sender, recipient and message details) to the interaction diagram notation.

 **DEFINITION: Interaction Diagram Notation:** The notation used for interaction diagrams is standard. Time increases as one moves down the page. Each agent has a *lifeline*, depicted as a vertical line with the name of the agent in a box at the top of the line. Messages are depicted as horizontal arrows between lifelines with a brief description of the message above the arrow. The simple example below shows two agents named *User* and *System* where the User sends a Query message to the System. This is followed by the System sending a Response message to the User.



 **TIP: Depicting Actions and Percepts:** Sometimes, showing actions and percepts on interaction diagrams can be important. For example, often an interaction is triggered by a percept. There are a number of ways that actions and percepts can be depicted in interaction diagrams. They can be shown as messages from an invisible lifeline (left in the example below), they can be shown as text on the lifeline of the relevant agent (centre in the example below), or they can be shown as messages to an explicit environment lifeline (right-hand side, below).



In the remainder of this section, we focus on the interesting step: determining where communication between agents is required. Wherever there is a step in the scenario that involves a functionality from a new agent, there must be some interaction from a previously involved agent to the newly participating agent. While it is not possible to automatically derive the interaction diagrams from the use case scenarios, substantial consistency checking is possible (see Section 4.5).

More generally, determining when communication between agents is needed (step (b) above) involves identifying the sequential dependencies between steps in the scenario. Whenever a step *n* must occur after step *m* (and the two steps are performed by functionalities that are not in the same agent), then there needs to be a message.

Analysing the sequential dependencies will identify where messages are required, and we then need to name the message type, and eventually develop a message descriptor for the type. Initially, it is usually useful to assume that each message in an interaction diagram (or protocol) is a new type (i.e. has a unique name). It may later be possible to coalesce some number of messages into a single implementation type, but for clarity of design information, they should usually be kept separate at this stage.

In many use case scenarios, the steps are sequential: step 2 must occur after step 1, step 3 must occur after step 2, and so on. However, this is not always the case. For example, looking at Figure 6.2 showing the end of the scenario where a user buys a book (Order Book scenario), the last three steps involve the Stock Manager logging the outgoing delivery (step 12 below), the customer relations agent registering the purchase in the user’s profile (step 13) and sending the customer an e-mail (step 14). Although the use case scenario presents these steps in a particular order, in fact they are not required to occur in that order.

Looking at Figure 6.2, we see that steps 10 and 11 are in the same agent (actually in the same functionality), so no message is required. We now consider a number of possible dependencies in the remaining steps.

Option 1: The steps are required to be in the given sequence, that is, step 12 must occur after step 11, step 13 after 12 and step 14 after step 13.

In this case, we would clearly need messages from Delivery Manager to Stock Manager, and Stock Manager to Customer Relations. We might consider that as steps 13 and 14 are within the same agent, no message is required here. This would require the Profile Monitor functionality to tell the Customer Contact functionality to e-mail the customer when it has finished registering the purchase. However, this design decision would not be a good one: this scenario is dealing with ordering books, and the Profile Monitor should not be responsible for telling the Customer Contact functionality that it

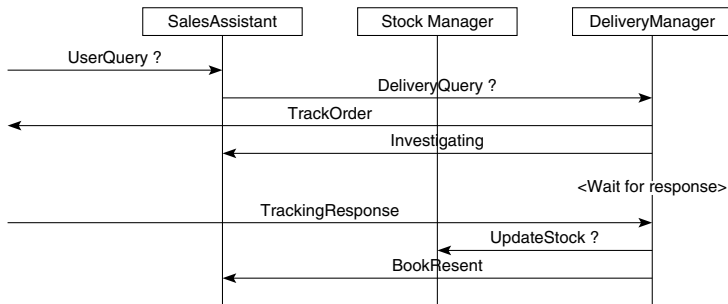
	Step	Functionality	Agent
⋮			
10.	Place delivery request	Delivery Handling	Delivery Manager
11.	Log outgoing delivery	Delivery Handling	Delivery Manager
12.	Log books outgoing	Stock management	Stock Manager
13.	Update Customer Profile	Profile Monitor	Customer Relations
14.	Send email	Customer Contact	Customer Relations

Figure 6.2 Final steps in a scenario, showing also agent information

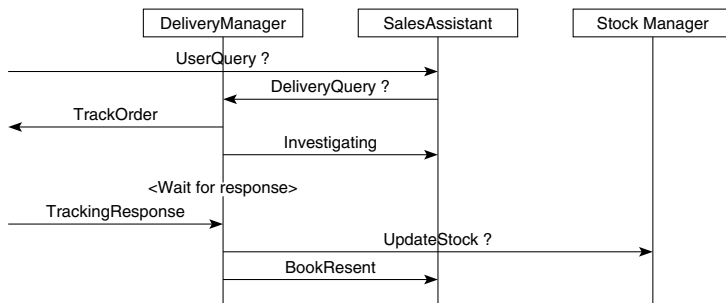
should let the user know that their order has been sent. Consequently, we require the Delivery Manager to notify the Customer Relations agent when it is time to send e-mail to the customer. This requires the addition of a reply message to the Delivery Manager, indicating when the profile updating is completed, as is shown in Figure 6.3.

Option 2: Steps 12, 13 and 14 must all occur after step 11, but can occur in any order. In this case, after step 11, messages are sent from the Delivery Manager to the other two agents, as shown in Figure 6.4. Because the interaction diagram captures a particular ordering, showing of this potential parallelism is left for the interaction protocol. The interaction diagram simply captures the same (arbitrary) ordering as that shown in the use case scenario. However, establishing whether there is a need for strict sequencing can effect what messages are necessary, as we have seen.

 **TIP:** In some cases, hard-to-read interaction diagrams can be improved by changing the order of the agents. For example, the following diagram is not very readable.



However, by changing the order of agents, we obtain a more readable diagram:



 **TIP: Including the trigger:** The table of steps in the use case scenario may not include the trigger. When developing the interaction diagrams, consider whether it is useful to include the triggering message or percept within the interaction diagram.

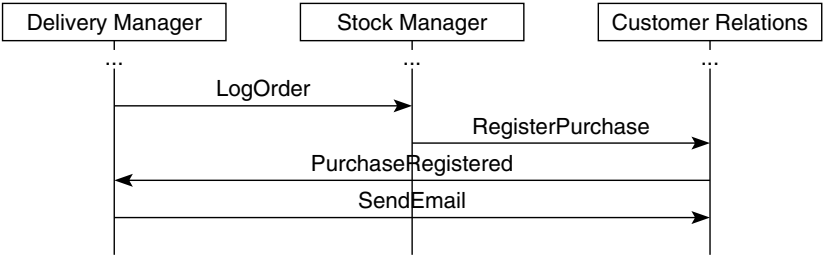


Figure 6.3 Option 1, interaction diagram for sequential steps in scenario

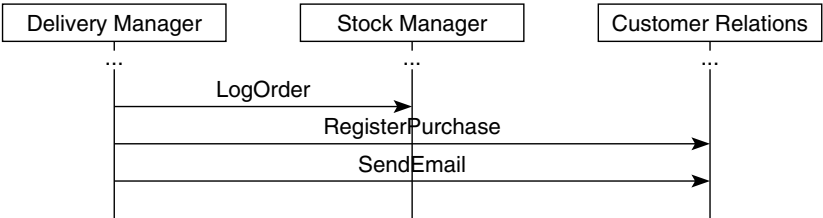


Figure 6.4 Option 2, interaction diagram where strict sequencing from scenario was not necessary

Following is a larger worked example from the Electronic Bookstore.

Electronic Bookstore: Case study

Here is the outline of the **Query Late Books Scenario** developed in the first round of scenario descriptions:

Query Late Books Scenario
Trigger: User query (via Online Interaction)

Key for functionalities:

DH Delivery Handling
LGM Lost Goods Management
CC Customer Contact
SM Stock Management

	Step type	Step	Functionality
1	GOAL:	Determine delivery status	DH
2	GOAL:	Log delivery problem	LGM
3	ACTION:	Request delivery tracking	LGM
4	GOAL:	Inform customer	CC

5	OTHER:	<i>Wait for response</i>	DH
6	PERCEPT:	<i>Tracking info received</i>	DH
7	GOAL:	<i>Arrange delivery</i>	DH
8	GOAL:	<i>Log books outgoing</i>	SM
9	GOAL:	<i>Inform customer</i>	CC
10	GOAL:	<i>Update delivery problem</i>	LGM

We begin by replacing functionalities with agents. We use numbers in brackets to indicate steps that occur in the same agent as the preceding step.

Key for agents:

DM: Delivery Manager

CR: Customer Relations

SM: Stock Manager

	Step type	Step	Agent
1	GOAL:	<i>Determine delivery status</i>	DM
(2)		<i>Log delivery problem</i>	DM
(3)		<i>Request delivery tracking</i>	DM
4	GOAL:	<i>Inform customer</i>	CR
5	delay ...		
6	PERCEPT:	<i>Tracking info received</i>	DM
(7)		<i>Arrange delivery</i>	DM
8	GOAL:	<i>Log books outgoing</i>	WM
9	GOAL:	<i>Inform customer</i>	CR
10	GOAL:	<i>Update delivery problem</i>	DM

We now consider the dependencies between these steps. Steps 1 to 3 are within an agent, and so we do not capture the dependencies between them at this level.

Step 4, informing the user that tracking has been requested, can be done in parallel with steps 1 to 3, or could be required to be done only after the delivery tracking has actually been requested. In this case, we assume the latter: the system will wait until tracking has been requested before telling the user that it is investigating. By showing step 3 on the interaction diagram, we can show that step 4 must occur after the tracking is initiated.

Steps 5 (delay) and 6 (percept being received) must occur in sequence after step 4, and are thus shown in that order on the interaction diagram.

Step 7 (arrange delivery) is required to occur after step 6, but since it is internal to the Delivery Manager agent (which has just received the percept in step 6), no inter-agent communication is required.

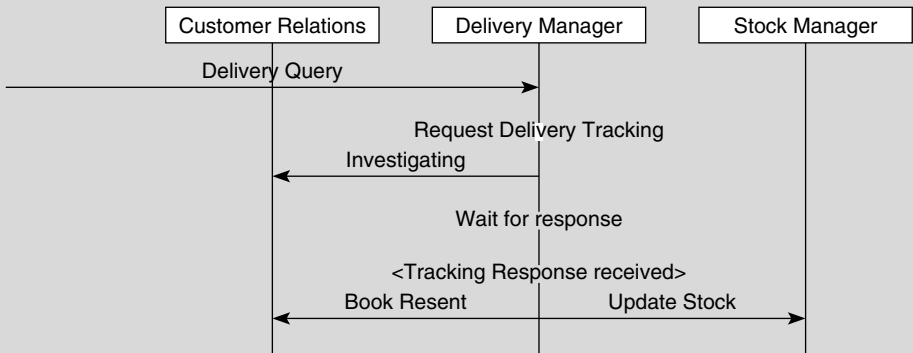
Step 8 (log books outgoing) is required to be after step 7 (arrange delivery), because we do not want to log books as being sent until we are certain that the delivery has been arranged and they will be sent. So, once delivery has been arranged (which is not shown in the interaction diagram), the Delivery Manager sends a message (*Update Stock*) to the Stock Manager, at which point the Stock Manager can log the outgoing books.

Step 9 (inform customer) is required to be after step 7 (but not necessarily after step 8): we do not want to tell the customer that their books have been resent until we know that they actually will be sent (i.e. after step 7), but we do not need to wait for the stock to be updated in order to tell the customer, so step 7 and step 8 can occur in parallel. Thus, after step 7 there the Delivery Manager sends a message (*Book Resent*) to the Customer Relations.

The two messages (triggering steps 8 and 9) can be sent simultaneously.

Finally, step 10 is required to be after step 7: we do not need to wait for the customer to be informed or for the stock records to be updated in order to update the delivery problem record, but we do want to wait for the delivery to be arranged. So, step 10 must come after step 7; since both steps are performed by the Delivery Manager, the dependency is internal and no message is required between agents.

This yields the following interaction diagram. Note that the trigger of the scenario is a *Delivery Query* message (possibly from Customer Relations or Sales Assistant) received by the Delivery Manager.



6.2 INTERACTION PROTOCOLS FROM INTERACTION DIAGRAMS

As with scenarios, we would expect to have only a representative set of interaction diagrams, not a complete set. In order to have complete and precisely defined interactions,

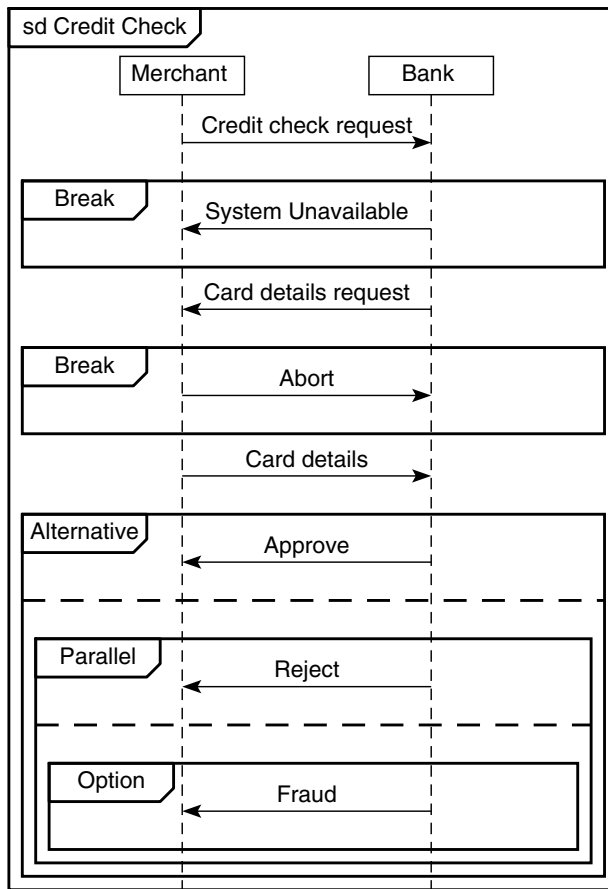


Figure 6.5 Protocol diagram for credit checking

we progress from interaction diagrams to protocols that define exactly which interaction sequences are valid within the system. Because protocols must show all variations, they are often larger than the corresponding interaction diagram and may need to be split into smaller chunks.

In order to give an example of developing a complete interaction protocol, we need a notation for describing interaction protocols. Since interaction protocols include sequencing, choices, iteration and other control structures, the notation needs to be quite expressive. A range of notations exist for describing protocols including UML activity diagrams, AUML (Agent UML) (Odell *et al.* 2000), and Petri nets (e.g. (Cost *et al.* 1999; Nowostawski *et al.* 2001; Poutakidis *et al.* 2002)). We will be using the revised version of Agent UML, (Huget *et al.* 2003) currently under development.

Developing protocols is done by considering alternatives. For each message (or percept) that an agent receives, we ask ‘what are the possible messages that the agent could

send as a response?’ We then repeat the process for these messages. More generally, we ask ‘what are the possible continuations (i.e. *sequences* of messages)?’

In some cases, the alternatives are best described in terms of parallelism or looping. For example, if an interaction diagram shows message m_1 followed by m_2 and an alternative is that m_2 is sent first followed by m_1 , then we could describe this as an alternative, but it is simpler to say that the two messages are sent in parallel.

For example, consider a credit card-checking protocol involving a merchant and a bank. The protocol starts with the merchant sending the bank a credit card request message. The bank will respond to this with a card details request; there are no alternative responses. The merchant, upon receiving the card details request, will respond with the card details; again there are no alternative responses. When the bank receives the card details, it can respond with a number of possible messages. One possibility is to respond with an *Approve*. Another possibility is to respond with a *Reject*. Finally, in some cases, the bank will respond with both a reject message (to the merchant) and also a *Fraud* message. Figure 6.5 shows this protocol. Note that sending *Reject* and *Fraud* can happen in either order or simultaneously.

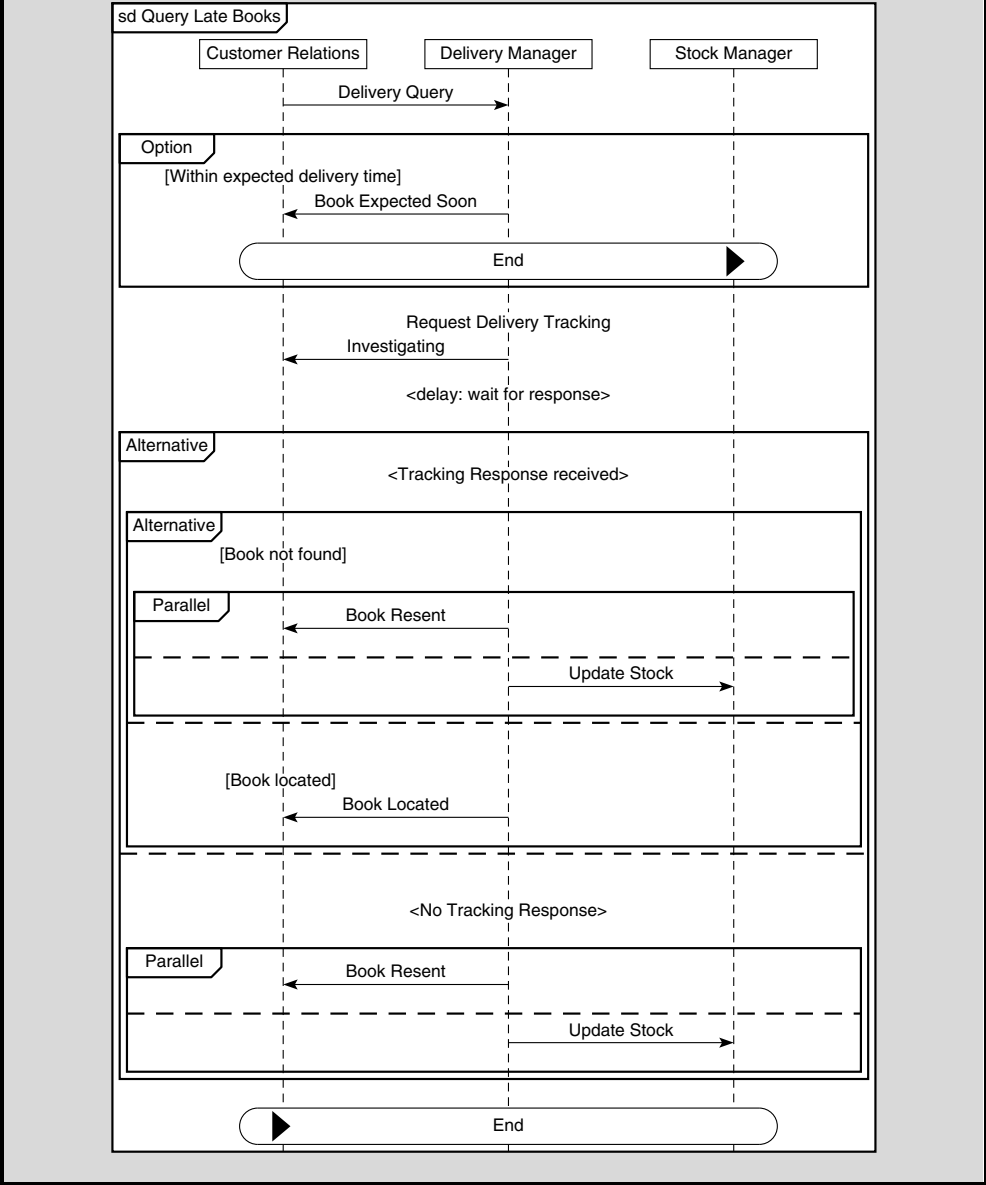
In considering possible alternatives, one source of possibilities is the variation field of the use case scenarios. Another source of possibilities is different use case scenarios. Often, there will be multiple scenarios that relate to the same concepts. For example, one scenario might show what happens when a customer orders a book that is in stock and another scenario might show what happens when a customer orders an out-of-stock book. Since interaction protocols are meant to cover all possibilities, we would consider both of these use case scenarios to be alternatives in a single interaction protocol.

Electronic Bookstore: Case study

The diagram below shows the protocol for the query late books use case scenario.

In developing it from the interaction diagram given earlier, we considered at each step what were the possible alternatives. In this case, we focus on the alternatives that occur in response to the tracking request. The use case scenario given in Chapter 4 outlines three variations to the case where the book is determined to be lost and is resent:

1. The book is within (or close to) the time window of expected delivery. In this case, no request is issued to track the order and the user is informed that the order is expected soon.
2. The tracking information received indicates that the book is on its way, thus there is no need for a new delivery.
3. The tracking request does not return any information (*NoTrackingResponse*). In this case, the book is resent.



6.3 DEVELOP PROTOCOL AND MESSAGE DESCRIPTORS

As with each of the final design entities in the system, we have descriptors for protocols and for messages. Descriptors allow us both to collect together information existing in other places and also to specify additional details about the particular entity.

The template that we use for the protocol descriptor is as follows:

- Name
- Description – brief natural language description
- Scenarios – lists the scenarios that are included in this protocol
- Agents – lists the agents involved in the protocol
- Messages – lists the messages involved in the protocol
- Notes – and other miscellaneous information.

During protocol development, a number of messages have been identified, which are the messages between agents in the system. In some cases, there are inter-agent messages that do not require a protocol, as they are a single message, or a message reply pair. If desired, these can be specified as degenerate protocols, but they can also be specified simply as messages.

In detailed design, there will also be additional *internal messages* added to the design.

Descriptors for messages clearly need to contain information about the content of the message. It is also useful to indicate the purpose of the message, which may be, for example, to transfer control to another agent or functionality, to request a service, or to update information.

For example, a Sales Assistant agent in the Electronic Bookstore may tell the Stock Manager that a book has been sold, with the primary purpose of updating the Stock Manager's records. The same Sales Assistant agent may send a message about the sale to a cashier agent with the primary purpose of transferring control to the cashier agent, which will then obtain necessary details for doing the financial transaction. Messages may of course perform multiple functions.

Fields in our template descriptor for messages are as follows:

- Name – brief identifier
- Description – natural language description containing any relevant information
- From agent
- To agent
- Purpose – for example, update of data, request service, transfer control, and so on
- Information carried – what information fields will this message carry.

Some additional fields will be described and added during detailed design.

Electronic Bookstore: Case study

An example message from the Electronic Bookstore is *Order Sent*, which is sent by the Delivery Manager agent to the Customer Relations agent when a pending order has been sent out.

- **Name** – Order Sent.
- **Description** – contains information about the contents of a pending order that has now been sent. It allows the sending of notification to the customer and updating of the customer record.
- **From agent** – Delivery Manager.
- **To agent** – Customer Relations.
- **Purpose** – to update the customer record and also to trigger sending an e-mail to the customer, if appropriate.
- **Information carried** – order reference, list of books sent, customer ID, date, carrier.